Part 1: Data from Raw to Ready

Data rarely arrive to us in a form such that they are ready for formal analysis. Typically, steps of preprocessing, cleaning, checking, etc. are required before we have what we can call **ready data**.

You will note that I avoid using the term **clean data**, as this can imply a complete lack of issues in the data set. Completely "clean" data is not realistic, and should not be expected. The best we can hope is that a data set is sufficiently processed to minimize any biases that may result from errors in the data set.

Ultimately, this process is quite situation-specific. We will present some key concepts and stress the importance of taking this step, but each data set will present unique challenges. This topic will be revisited throughout the course as we encounter new data sets.

Our discussion will initially focus on data sets that are naturally represented in a **flat** or **spreadsheet** form. Each column contains a different **variable** and each row gives one **instance** or **case**.

Exercise: Give examples of cases and associated variables in data sets typically encountered in finance.

When is a data set "ready?"

1. Each variable is properly named.
2. Each variable is understood.
3. Each variable is represented in its appropriate **form**.
4. Each variable has internal consistency, i.e., values have the same interpretation across all cases.
5. The source of any extreme values (outliers) is noted, and corrected as needed.
6. The source of any missing values is noted.

Example Data Set

First, we will read in a data set for use in some subsequent examples.

Visit the website

https://bit.ly/31QyYjK

and download the data from the first quarter of 2017. Create a directory for working on these examples, and place this file in that directory.

These data come from the U.S. Securities and Exchange Commission repository on market structure. This data set contains daily characteristics of a range of market variables over 5000 equities and ETFs, accumulated over several exchanges. See also

https://www.sec.gov/marketstructure/mar_methodology.html

Be sure that the working directory is correctly set in Python (use `chdir` from `os` if necessary) and use `read_csv` from `pandas`:

```
In [1]: import pandas as pd
        import matplotlib.pyplot as plt
        import numpy as np
        fulldata = pd.read_csv("q1_2017_all.csv")
```

Exercise: Visually inspect the data set and comment.

Background on the data can be found in the associated `README` file, available on Canvas in the "Data Sets" folder in the "Files" page. We will reference portions of it below.

Understanding the Variables

Whenever a new data set is encountered, one should fully understand its variables. Specific questions to address for each variable:

1. Describe the information that is stored in this column.
2. What type of variable is it? (Timestamp, categorical, ordinal, ratio, other?) Be sure that Python represents the variable correctly.
3. Are there any missing values? What is the source of the missingness?
4. Are there any extreme/inappropriate values? What is the source of the extreme value?

Exercise: Describe the distinctions between the variables which are on the categorical, ordinal, and ratio scales.

Column 1: Date

The first column in `fulldata` is a date stamp for the observation. Python/Pandas has special functions for handling dates which will prove useful later, including when working with time series.

After reading the file, the dates are simply integers.

```
In [2]: print(fulldata['Date'][0])
```

```
20170103
```

The function `to_datetime` in Pandas will make the conversion. The list on the following page shows all of the formatting characters.

```
In [3]: fulldata['Date'] = \
          pd.to_datetime(fulldata['Date'], \
                         format='%Y%m%d')
```

These strings can also be used in formatting output using `strftime`.

```
In [4]: print(fulldata.Date[0].strftime('%m/%y'))
        print(fulldata.Date[0].strftime('%A, %B %d'))
```

```
01/17
Tuesday, January 03
```

(From `https://stackabuse.com/how-to-format-dates-in-python/`.)

- `%b`: Returns the first three characters of the month name.
- `%d`: Returns day of the month, from 1 to 31.
- `%Y`: Returns the year in four-digit format.
- `%H`: Returns the hour.
- `%M`: Returns the minute, from 00 to 59.
- `%S`: Returns the second, from 00 to 59.
- `%a`: Returns the first three characters of the weekday, e.g. Wed.
- `%A`: Returns the full name of the weekday, e.g. Wednesday.
- `%B`: Returns the full name of the month, e.g. September.
- `%w`: Returns the weekday as a number, from 0 to 6, with Sunday as 0.
- `%m`: Returns the month as a number, from 01 to 12.
- `%p`: Returns AM/PM for time.
- `%y`: Returns the year in two-digit format, that is, without the century.
- `%f`: Returns microsecond from 000000 to 999999.
- `%Z`: Returns the timezone.
- `%z`: Returns UTC offset.
- `%j`: Returns the number of the day in the year, from 001 to 366.
- `%W`: Returns the week number of the year, from 00 to 53, with Monday being counted as the first day of the week.
- `%U`: Returns the week number of the year, from 00 to 53, with Sunday counted as the first day of each week.
- `%c`: Returns the local date and time version.
- `%x`: Returns the local version of date.
- `%X`: Returns the local version of time.

Columns 2 and 3: Security and Ticker

The variables `Security` and `Ticker` are currently of type `object`, which is how Pandas stores strings.

```
In [5]: fulldata[['Security','Ticker']].dtypes

Out[5]: Security    object
        Ticker      object
        dtype: object
```

Both of these variables are clearly categorical, and lacking in a natural ordering to the levels. The conversion to the appropriate data type is shown below. (By default, the categories are taken to be unordered.)

```
In [6]: fulldata['Security'] = \
          fulldata['Security'].astype('category')
        fulldata['Ticker'] = \
          fulldata['Ticker'].astype('category')

        fulldata[['Security','Ticker']].dtypes

Out[6]: Security    category
        Ticker      category
        dtype: object
```

We can see that there are two possible levels for `Security`, `Stock` or `ETF` (Exchange Traded Fund). There are over 5,000 symbols under consideration.

```
In [7]: print(fulldata['Security'].unique())
        print(len(fulldata['Ticker'].unique()))
```

```
[Stock, ETF]
Categories (2, object): [Stock, ETF]
5338
```

Exercise: How many of the different ticker symbols are ETFs?

Exercise: Execute the commands below, and discuss what you find.

```
In [8]: len(fulldata['Date'].unique())
        fulldata.groupby('Ticker')['Date'].\
          count().value_counts().sort_index()
        fulldata.groupby('Ticker')['Date'].\
          count().idxmax()
        fulldata[fulldata.duplicated(\
          subset=('Date','Ticker'),keep=False)]
        None
```

We can remove the duplicated rows using the following command.

```
In [9]: fulldata = fulldata.drop_duplicates(\
          subset=('Date','Ticker'), keep='last')
```

Columns 4 through 7: Rank Variables

The next four columns provide ranks of the stocks/ETFs with respect to key attributes of Market Capitalization, Turnover, Volatility, and Price.

In general, we would prefer to start with data in its most "raw" format, and one could construct variables such as these from that data. Alas, we are often left to work with what is avaiable.

Exercise: Inspect the output of

```
In [44]: fulldata.groupby(by=['Date','Security',\
             'VolatilityRank'])['Security'].count()
         fulldata.groupby(by=['Security','Ticker',\
             'VolatilityRank'])['VolatilityRank'].count()
         None
```

What does this tell us about the structure of these variables? Why should we be very careful with these variables?

We change these into **ordered categorical variables** to reflect their ordinal nature.

```
In [11]: fulldata = fulldata.copy()
         fulldata['McapRank'] = fulldata['McapRank'].\
           astype('category').cat.as_ordered()
         fulldata['TurnRank'] = fulldata['TurnRank'].\
           astype('category').cat.as_ordered()
         fulldata['VolatilityRank'] = \
           fulldata['VolatilityRank'].\
             astype('category').cat.as_ordered()
         fulldata['PriceRank'] = fulldata['PriceRank'].\
           astype('category').cat.as_ordered()
```

Columns 8 through 19: The Count Variables

The remaining column in the data set consist of trade and share counts of different types.

Note that some of the volume counts are reported in 1000's of shares, hence there are non-integer values among the counts.

Exercise: Execute the command below and comment on the results. It is especially important to focus on extreme or impossible values.

```
In [12]: fulldata.describe()
```

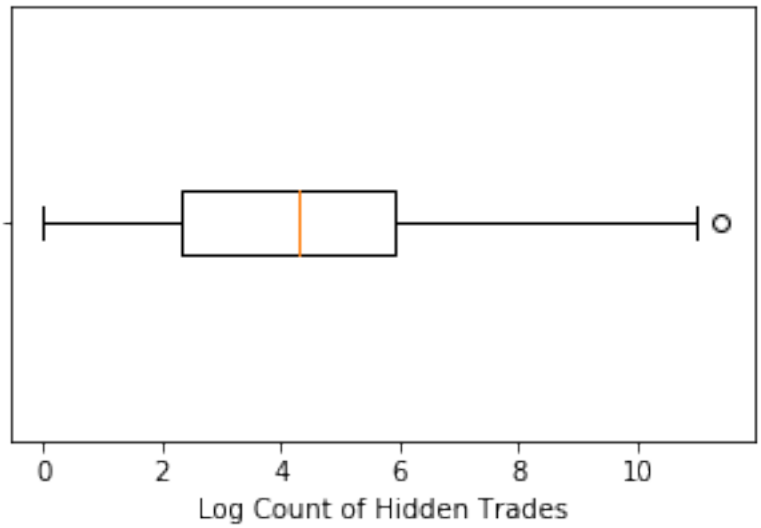
```
None
```

Of course, graphical tools are useful for understanding the properties of variables. This topic will be covered more fully later, but here we consider a classic approach: the **boxplot**:

```
In [13]: plt.figure(figsize=(5,3))
         plt.boxplot(np.log(fulldata['Hidden'])\
              .dropna(),labels=[""], vert=False)
         plt.xlabel("Log Count of Hidden Trades")
         plt.show()
```

```
/Users/chadschafer/anaconda3/lib/python3.7/site-packages/ipyker
```

```
/Users/chadschafer/anaconda3/lib/python3.7/site-packages/ipyker
```



The "box" of the plot extends from the 25th to the 75th percentile of the data, while the solid line in the middle of the box is at the median. The two "arms" extend to the minimum and maximum value, **except** that any value labelled an **outlier** is plotted separately.

Exercise: How is an outlier defined in this case?

Exercise: Why was the logarithm of the variable utilized? What issues did that create?

Exercise: Consider the outlier in the previous plot. Is this observation also extreme in the other variables? Can you find the source of this behavior?

Exercise: Parse the syntax, and comment on the output, of the following command.

```
In [14]: fulldata.iloc[:, 7:19].\
             apply(lambda x: x.idxmax())
         None
```

Missing Data

Most data sets contain some amount of missing data, for many reasons.

As a first step, it is important to recognize how your data set represents missing values, so that they are read in properly.

In a `.csv` file such as that we are using here, it is common to simply have blank values to indiciate missingness, i.e., there are consecutive commas without a value between.

The `read_csv()` function has an argument `na_values` which allows the user to specify strings that should be considered as missing values. By default, the following are converted into `NaN`: `, #N/A, #N/A N/A, #NA, -1.#IND, -1.#QNAN, -NaN, -nan, 1.#IND, 1.#QNAN, N/A, NA, NULL, NaN, n/a, nan, null`. **Be careful:** It is not unusual for data sets to use numbers such as `-999` to indicate a missing value.

Pandas uses `NaN` to represent missing values.

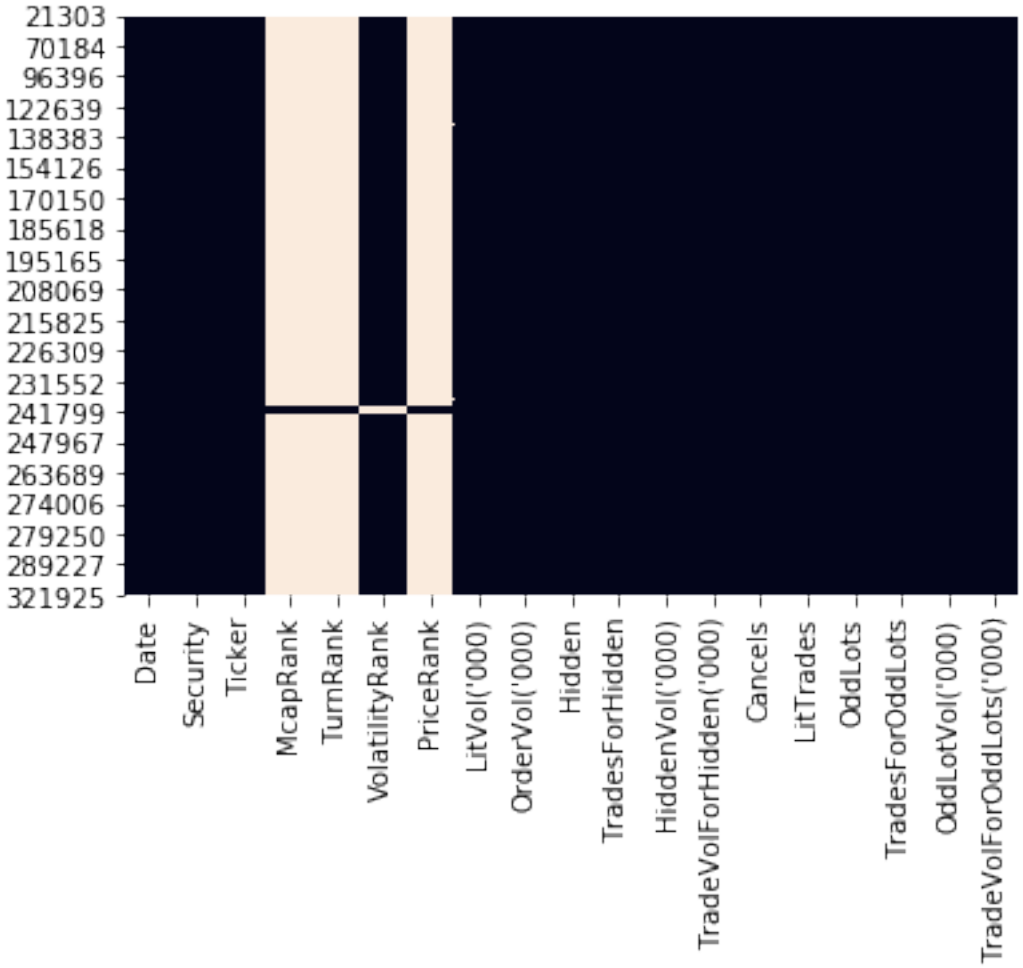
Exercise: Execute the following commands and discuss the results

```
In [15]: np.where(fulldata.isnull())
         len(np.unique(np.where(fulldata.isnull())[0]))
```

```
Out[15]: 77
```

The package `seaborn` has a function `heatmap()` that can be useful for inspecting any patterns in the missingness.

```
In [16]: import seaborn as sns
         sns.heatmap(fulldata.iloc[np.unique\
             (np.where(fulldata.isnull())[0]),:]\
             .isnull(),cbar=False)
         None
```



The `dropna` function will remove any row from the data frame with at least one `NaN` value.

```
In [17]: fulldatacomplete = fulldata.dropna()
         len(np.unique(np.where(fulldatacomplete.\
                 isnull())[0]))
```

```
Out[17]: 0
```

Working only with such data is referred to as **complete case analysis**.

Exercise: Discuss potential drawbacks to complete case analyses.

Exercise: Consider the output of the following and comment on any features of the incomplete cases that may be releveant.

```
In [18]: fulldata.iloc[np.unique(np.where(fulldata.\
             isnull())[0]),:]
```

None